

00 · Guía paso a paso

Quant-Backtester · material de estudio

Este proyecto es un motor de backtesting vectorizado en pandas que vas a construir tú. No partes de cero: la "fontanería" (descarga de datos, validaciones, orquestación del motor, tablas, gráficos, notebooks) ya está hecha. Lo que falta es justo la materia que se quiere aprender: 26 funciones marcadas como **TODO** que lanzan `NotImplementedError` hasta que las escribas.

Tres piezas te acompañan:

- **Los tests** (`tests/`) son tu autocorrector. Están completos y comprueban las convenciones exactas de cada TODO.
- **Los PDF** (`docs/`) explican la teoría, la notación y cómo pasar de la fórmula al código, con pseudocódigo y pistas graduales. No contienen soluciones.
- **Los notebooks** (`notebooks/`) usan lo que vas construyendo con datos reales y te hacen las preguntas que importan.

1. Preparar el entorno

Necesitas Python 3.11 o superior. Desde la raíz del repo, crea el entorno virtual, actívalo e instala las dependencias:

```
python -m venv .q-backtester

# PowerShell
.q-backtester\Scripts\Activate.ps1
# Git Bash
source .q-backtester/Scripts/activate

python -m pip install -r requirements.txt
python -m pip install -r requirements-dev.txt    # opcional: regenerar los PDF
```

En VS Code, elige el intérprete del entorno (`Python: Select Interpreter` → `.q-backtester`) y úsalo también como kernel de los notebooks.

Un entorno virtual no se puede mover

El venv guarda rutas absolutas. Si mueves o renombras la carpeta del proyecto (por ejemplo, al sacarla de OneDrive), `python.exe` sigue funcionando, pero los lanzadores `pytest.exe`, `jupyter.exe` y `pip.exe`, y también `activate.bat`, apuntan a la ruta antigua y fallan. Hay dos soluciones: recrear el entorno (borra `.q-backtester` y repite los pasos de arriba) o usar siempre `python -m pytest`, `python -m pip` y `python -m jupyter`, que no dependen de esas rutas.

No trabajes dentro de OneDrive

Un venv son miles de ficheros pequeños. OneDrive intenta sincronizarlos, bloquea algunos mientras los sube y puede dejar el entorno a medias. Guarda el proyecto en una carpeta local (por ejemplo `C:\Users\tu_usuario\Proyectos`) y usa git para la copia de seguridad.

Smart App Control y pyarrow

Si Windows tiene activado Smart App Control, puede bloquear DLL recién publicadas ("Una directiva de Control de aplicaciones bloqueó este archivo"). Pasa con pyarrow 25.0.x, que es lo que se usa para leer y escribir parquet. Por eso `requirements.txt` pide `pyarrow<25`. Para comprobarlo, ejecuta `python -c "import pyarrow.parquet"`: si no da error, todo está bien.

Después, descarga los datos y lanza los tests por primera vez:

```
python -m src.data      # descarga los 10 tickers desde 2010 a data/raw/prices.parquet
python -m pytest        # primera ejecución
```

El resultado esperado es algo así como `32 passed, 131 skipped`, seguido de una sección **TODO pendientes** con la lista de los TODO que bloquean algún test. Los 32 tests que pasan comprueban la fontanería. Los 131 *skipped* esperan a tus TODO.

2. Cómo trabajar cada TODO

Repite este ciclo para cada TODO, en orden:

1. Lee la sección del PDF correspondiente: la intuición, la fórmula y las convenciones.
2. Abre la función en `src/`. Lee el docstring entero y los comentarios `# TODO`: ahí está el contrato exacto.
3. Sustituye la línea `raise NotImplementedError(...)` por tu implementación.
4. Lanza el checkpoint de esa función: `python -m pytest -k nombre_funcion`.
5. Si algo falla, lee el mensaje del test: casi todos explican qué esperaban y por qué. Corrige y repite.
6. Cuando esté en verde, marca la casilla en `docs/progreso.md` y haz un commit.

Prueba con tablas pequeñas

Antes de lanzar los tests, prueba la función en un notebook o en la consola con una tabla de 4 o 5 filas que puedas calcular a mano. `tests/helpers.py` tiene `frame({"A": [... ]})`, que crea un DataFrame con fechas hábiles en una línea.

Algunas reglas del juego:

- **No modifiques los tests.** Si crees que un test está mal, probablemente hay una convención que se te ha escapado. Vuelve al docstring.
- Un TODO pendiente aparece como *skipped*, no como fallo. Para ver el motivo de cada uno, añade `-rs`. Si quieres que cuenten como fallos (por ejemplo, para comprobar que lo has terminado todo), usa `--todo-fail`.
- Los tests convierten en error cualquier aviso de API obsoleta de pandas lanzado desde `src/` (`FutureWarning` o `DeprecationWarning`). Si te sale uno, el mensaje te dice qué usar en su lugar.
- Algunos tests de integración necesitan varios TODO a la vez. Por ejemplo, `run_backtest` usa todo el motor. En la lista de pendientes verás siempre el *primer* TODO que bloquea cada test.

3. Mapa de los TODO

TODO	Función	Fichero	Teoría	Checkpoint
1.1	simple_returns	src/data.py	01	-k simple_returns
1.2	log_returns	src/data.py	01	-k log_returns
1.3	clean_prices	src/data.py	01	-k clean_prices
2.1	signals_to_positions	src/backtest.py	02	-k signals_to_positions
2.2	compute_turnover	src/backtest.py	02	-k compute_turnover
2.3	compute_costs	src/backtest.py	02	-k compute_costs
2.4	asset_pnl	src/backtest.py	02	-k asset_pnl
2.5	to_portfolio	src/backtest.py	02	-k to_portfolio
2.6	equity_curve	src/backtest.py	02	tests/test_backtest.py
3.1	sma_crossover	src/strategies.py	03	-k sma_crossover
3.2	momentum_12m	src/strategies.py	03	-k momentum_12m
3.3	bollinger_mean_reversion	src/strategies.py	03	-k bollinger
4.1	annualized_volatility	src/metrics.py	04	-k annualized_volatility
4.2	cagr	src/metrics.py	04	-k cagr
4.3	sharpe_ratio	src/metrics.py	04	-k sharpe
4.4	sortino_ratio	src/metrics.py	04	-k sortino
4.5	drawdown_series	src/metrics.py	04	-k drawdown_series
4.6	max_drawdown	src/metrics.py	04	-k max_drawdown
4.7	calmar_ratio	src/metrics.py	04	-k calmar
4.8	hit_ratio	src/metrics.py	04	-k hit_ratio
4.9	annual_turnover	src/metrics.py	04	-k annual_turnover
5.1	generate_windows	src/walkforward.py	05	-k generate_windows
5.2	evaluate_params	src/walkforward.py	05	-k evaluate_params
5.3	grid_search	src/walkforward.py	05	-k grid_search
5.4	walk_forward	src/walkforward.py	05	-k walk_forward
6.1	apply_vol_target (extensión)	src/backtest.py	06	-m extension

Al terminar cada bloque, lanza el fichero de tests completo: `python -m pytest tests/test_data.py, tests/test_backtest.py, tests/test_strategies.py, tests/test_metrics.py y tests/test_walkforward.py. tests/test_quantstats_check.py` comprueba tus métricas contra la librería `quantstats`. El capítulo **A** es una chuleta de pandas para series temporales: tenla a mano todo el rato.

4. Plan de tres días y flujo de git

El repo tiene una rama por día: `feature/datos`, `feature/estrategias` y `feature/walk-forward`. Las tres parten del mismo commit que `main`, el del andamiaje.

Día	Rama	TODO	Notebook	Al terminar
1	feature/datos	1.1 - 1.3 y 2.1 - 2.6	01_exploracion	tests de datos y motor en verde

Día	Rama	TODO	Notebook	Al terminar
2	<code>feature/estrategias</code>	3.1 – 3.3 y 4.1 – 4.9	<code>02_estrategias</code>	tabla comparativa con y sin costes
3	<code>feature/walk-forward</code>	5.1 – 5.4 (y 6.1 si sobra tiempo)	<code>03_walkforward</code>	resultados en el README

Al cerrar cada día, integra la rama en `main` y lleva `main` a la rama del día siguiente:

```
git switch feature/datos
git add src/data.py src/backtest.py docs/progreso.md
git commit -m "Datos y motor: TODO 1.x y 2.x"

git switch main
git merge --ff-only feature/datos           # main avanza hasta tu trabajo del día 1

git switch feature/estrategias
git merge main                             # el día 2 empieza con el motor ya hecho
```

Haz commits pequeños y frecuentes, por ejemplo uno por TODO o por grupo de TODO. Si un día se alarga, no pasa nada: los días son una guía, no un examen.

5. Cómo leer los tests

Cada fichero de `tests/` corresponde a un módulo de `src/`. Los nombres de los tests contienen el nombre de la función que comprueban, por eso funciona `-k`. Están ordenados igual que los TODO y suelen ir de lo simple a lo sutil:

- **Valores conocidos.** Ejemplos pequeños calculados a mano (por ejemplo, `test_compute_turnover_known_values`). Son los primeros que debes hacer pasar.
- **Convenciones.** Detalles que deciden el resultado exacto: la primera fila, los NaN, la división entre N, el capital inicial como pico.
- **Propiedades.** Cosas que deben cumplirse siempre, como la causalidad: "cambiar precios futuros no cambia señales pasadas".
- **Integración.** Varias piezas juntas, como los dos tests del README o el walk-forward completo.

Las fixtures generan precios sintéticos sin red y siempre iguales (con semilla fija): un paseo aleatorio geométrico (`gbm_prices`), una serie que revierte a la media (`ou_prices`), tendencias en línea recta (`trend_prices`) y once años con un régimen alcista o bajista por año (`regime_prices`). Están en `tests/helpers.py` y `tests/conftest.py`.

Qué hace el autocorrector por dentro

`tests/conftest.py` intercepta cada test que termina en `NotImplementedError` y lo marca como *skipped* con el mensaje del TODO. Al final de la ejecución agrupa esos mensajes en la sección "TODO pendientes", ordenados por número. El primero de la lista es tu siguiente tarea.

6. Si te atascas

1. Vuelve a leer el docstring: la mayoría de los atascos son una convención que no se ha leído con calma.
2. Lee las pistas del PDF en orden. La Pista 1 es conceptual y la 3 casi te da el detalle.
3. Consulta la chuleta de pandas (capítulo A): `shift`, `rolling`, `ffill`, `mask`, `.loc` frente a `.iloc`...
4. Reduce el problema: una columna, cinco filas, valores redondos. Imprime cada paso intermedio.
5. Pide ayuda concreta, por ejemplo a Claude: "dame una pista para el TODO 3.3 sin darme la solución" funciona mejor que "hazlo por mí".

7. Checklist

Lleva el progreso en `docs/progreso.md`, que puedes marcar directamente en VS Code. Cuando todo esté en verde, `python -m pytest --todo-fail` debería terminar sin fallos ni *skipped*, salvo los de quantstats si no lo tienes instalado. Ese es el momento de rellenar la sección **Resultados** del README con la tabla y el gráfico que genera el notebook 03.